Matching in Bipartite Graphs (2)

Peerapon S.

CPE 231 Algorithms

Topics

Maximum-weight matching

One-to-many maximum matching

Scenario I: Task Assignment

Hours spent by each group member on each task

	T1. Design algorithm	T2. Write/ debug code	T3. Analyze complexity and correctness	T4. Do report & presentation
P1	9	2	7	8
P2	6	4	3	7
P3	5	8	1	8
P4	7	6	9	4

Goal: Assign tasks to persons to ***minimize*** the total hours.

Solution from Brute-force algorithm:

Consider all combinations.

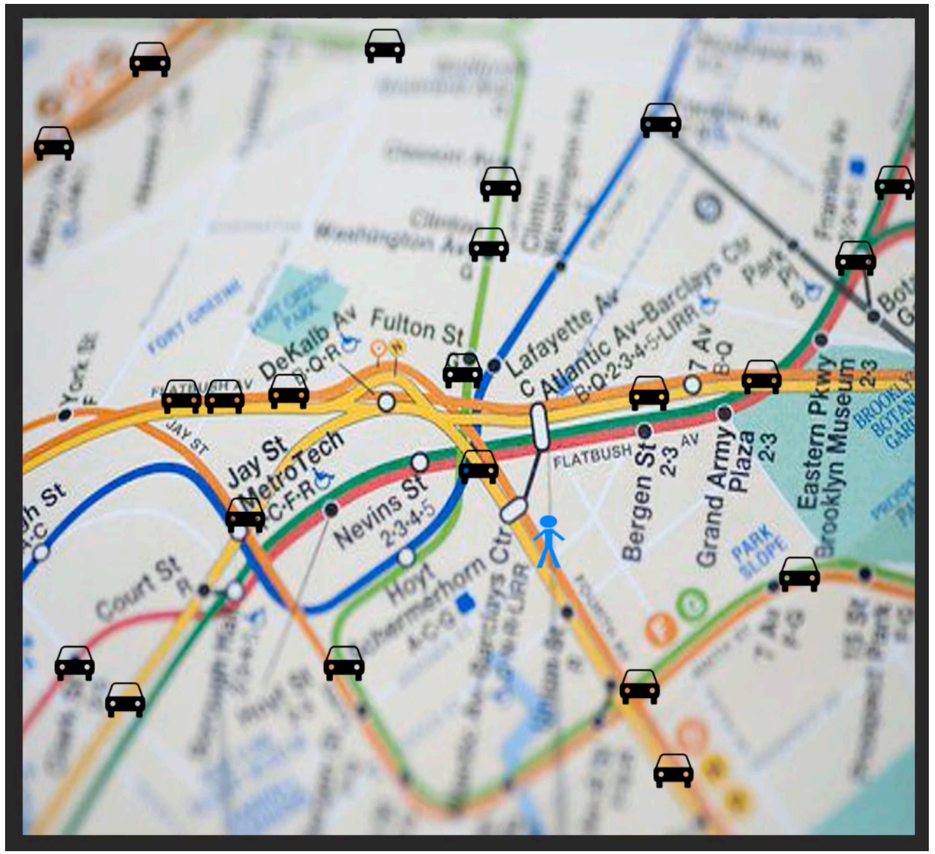
How many of them if you have 10 tasks and 10 persons ?

What is the efficiency of the brute-force algorithm?

Scenario II: Driver-Rider Matching

N drivers and *N* riders in a given area.

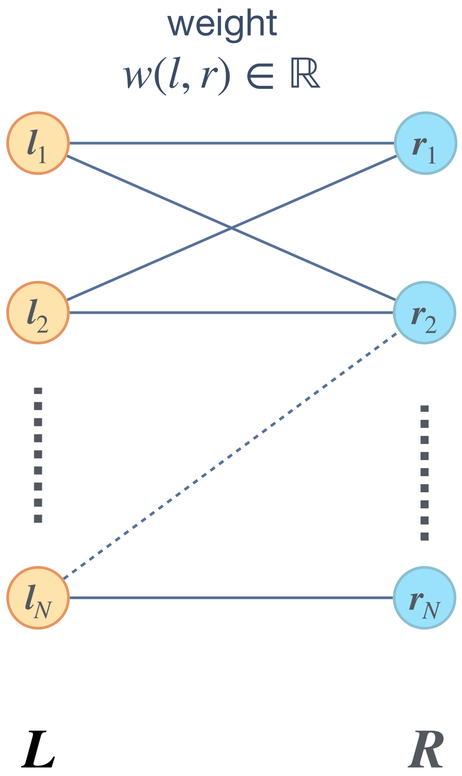
How to match drivers to riders so that the total wait time is minimized?



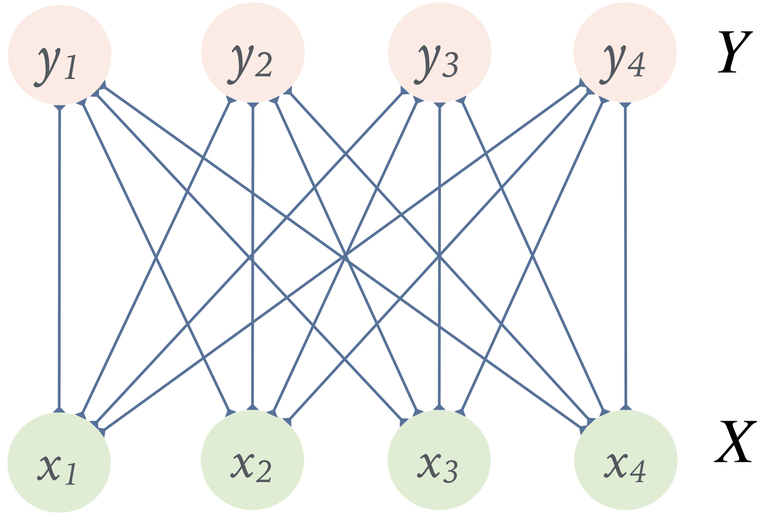
	D1	D2	...	D20
R1	9	2	...	8
R2	6	4	...	7
...	...	...	...	...
R20	7	6	...	4

Weighted Complete Bipartite Graph

A complete bipartite graph $G = (V, E)$ where $V = X \cup Y$, $|X| = |Y|$, and edge weights $w(x, y), (x, y) \in E$.



weight $w(x, y) \in \mathbb{R}$

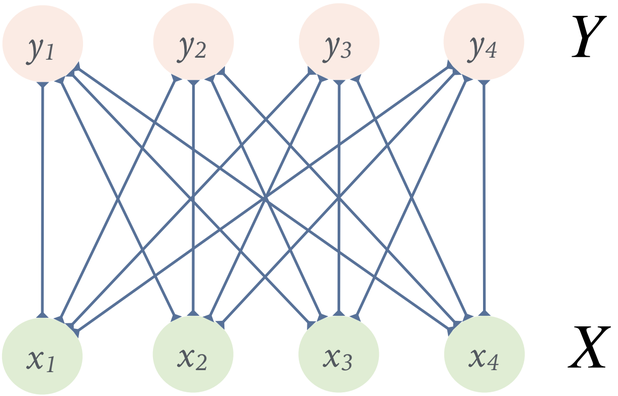


Maximum-Weight Bipartite Matching (Assignment Problem)

Let $w(M) = \Sigma_{(x,y)\in M} w(x, y)$ be the weight of the matching M.

Find a perfect matching M^* such that

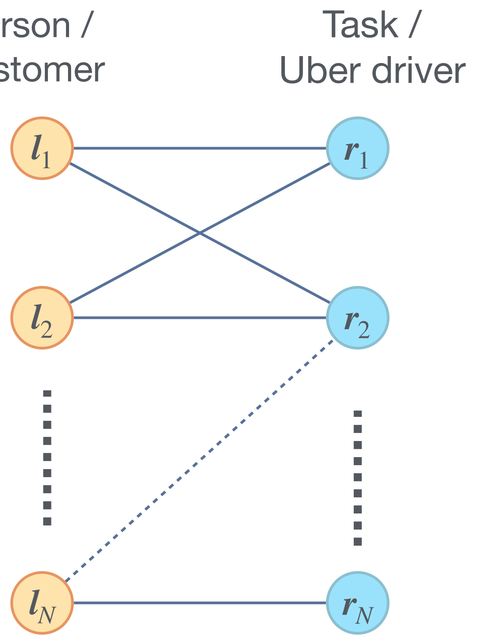
$$w(M^*) = \max\{w(M) : \ M \text{ is a perfect matching}\}$$



For the minimization objective, modify the input graph G with weights $w(e)$ to graph G' with weights $w'(e)$:

$$C = \max_{e \in E} w(e) + 1$$

$$w'(e) = C - w(e)$$



Weights = Task Time or Waiting time

Original input

	D1	D2	D3	D4
P1	9	2	7	8
P2	6	4	3	7
P3	5	8	1	8
P4	7	6	9	4

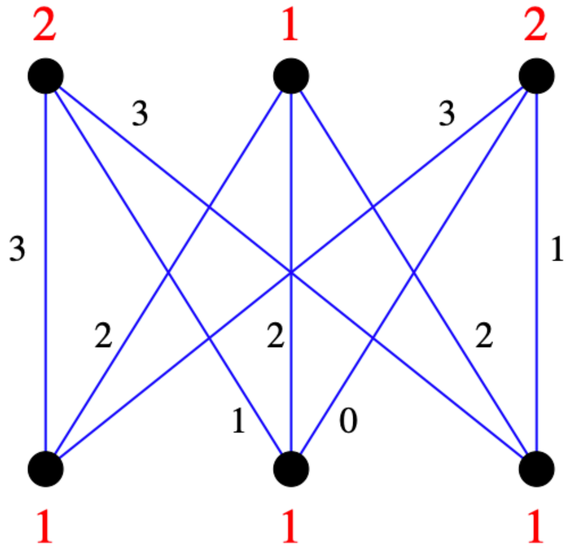
Input to maximum-weight matching

	D1	D2	D3	D4
P1				
P2				
P3				
P4				

Feasible Vertex Labeling

Vertex labeling of G is a **function** $l : V \to \mathbb{R}$

Feasible vertex labeling: $l(x) + l(y) \geq w(x, y),\ \forall x \in X,\ y \in Y$



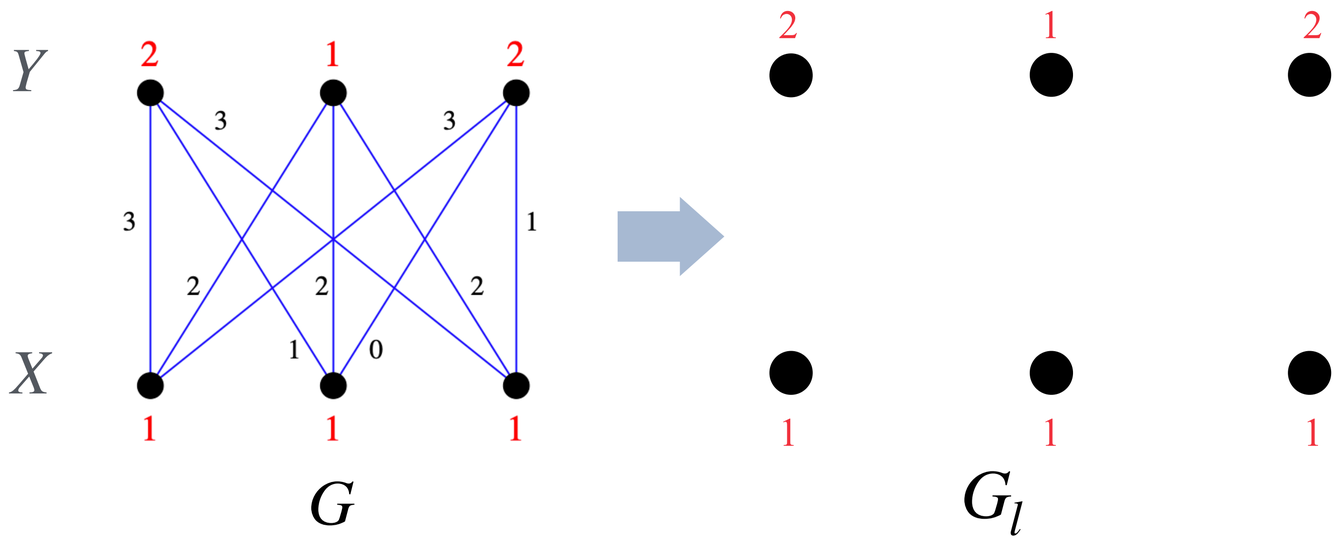
Default labeling:

$$l(x) = \max\{w(x, y) : y \in Y\} \qquad \forall x \in X$$
$$l(y) = 0 \qquad \forall y \in Y$$

The weight of ***any matching*** in G is always at most the sum of vertex labels.

Equality Subgraph

For feasible labeling l, define $G_l = (V, E_l)$, where $E_l = \{(x, y) \in E : \ l(x) + l(y) = w(x, y)\}$



The weight of perfect matching in G_l is equal to the sum of vertex labels.

With appropriate vertex labeling, any perfect matching in G_l is also a maximum-weight perfect matching in G.

Theorem [Kuhn-Munkres]: If l is feasible labeling and $\boldsymbol{M}^*$ is a perfect matching in equality subgraph $\boldsymbol{G}_l$, then $\boldsymbol{M}^*$ is a maximum-weight matching in $\boldsymbol{G}$.

Proof of Kuhn-Munkres Theorem

Consider a perfect matching M^* in the equality subgraph G_l: $w(M^*) = \sum_{(x,y)\in M^*} w(x,y)$

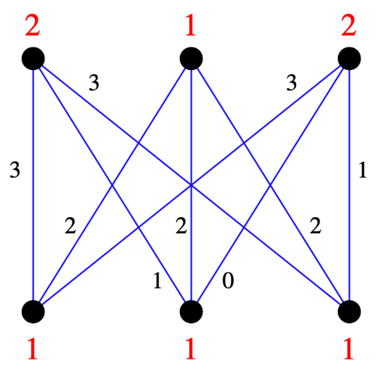
$$= \sum_{(x,y)\in M^*} l(x) + l(y)$$

For any matching M in G, since $l(x) + l(y) \geq w(x,y)$, the weight sum is always at most the sum of vertex labels:

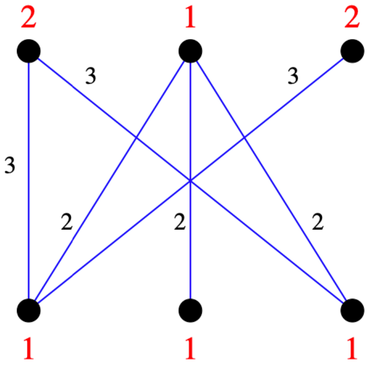
$$w(M) = \sum_{(x,y)\in M} w(x,y)$$

$$\leq \sum_{(x,y)\in M} l(x) + l(y) = w(M^*)$$

So, M^* is the maximum-weight perfect matching in G.



G



G_l

Hungarian (Kuhn-Munkres) Algorithm

Algorithm Hungarian (Kuhn-Munkres)

Input: A weighted complete bipartite graph $G = (V, E), V = X \cup Y, |X| = |Y| = n$
Output: A maximum-weight perfect matching M in G

Obtain G_l from G by using default vertex labeling l.
Initialize any arbitrary matching M (e.g., empty matching, greedy matching) and derive directed equality subgraph $G_{M,l}$

while M is not perfect matching in G_l **do** ← At most n steps
    P = Find-Augmenting-Path $(G_{M,l})$ ← $O(n^2)$
    $M = M \oplus P$
    Update G_l and derive $G_{M,l}$
return M

Add dummy vertices and zero weights if $|X| \neq |Y|$.

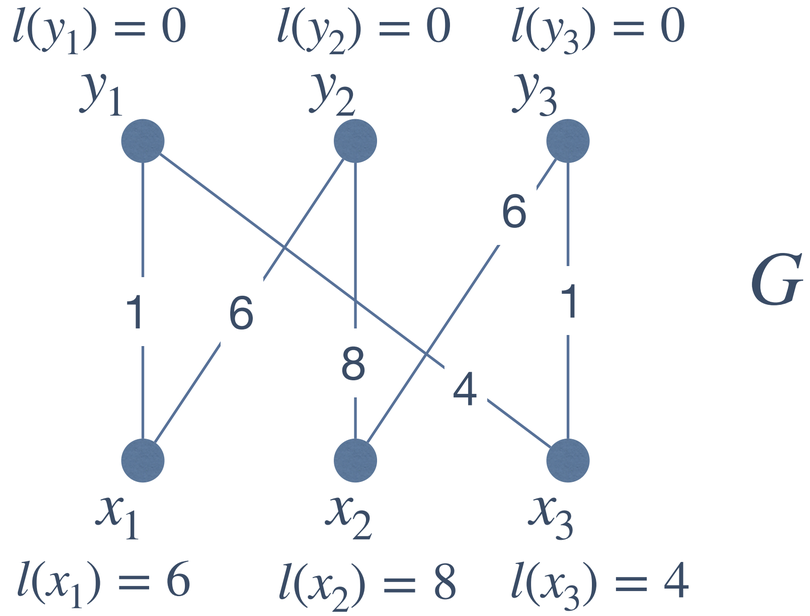
Greedy Maximal Bipartite Matching

```
GREEDY-BIPARTITE-MATCHING(G)
1  M = ∅
2  for each vertex l ∈ L
3      if l has an unmatched neighbor in R
4          choose any such unmatched neighbor r ∈ R
5          M = M ∪ {(l, r)}
6  return M
```

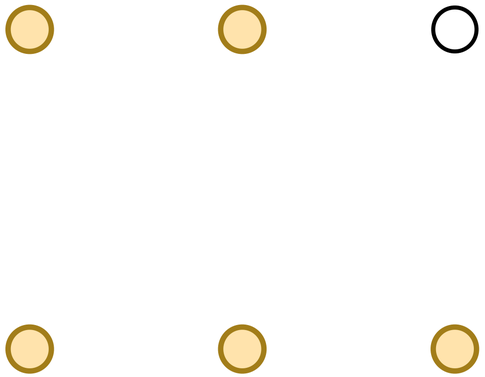
① **Do feasible labeling and derive G_l**

$$l(y) = 0, \forall y \in Y$$
$$l(x) = \max_{y \in Y} \{w(x, y)\}, \ \forall x \in X$$

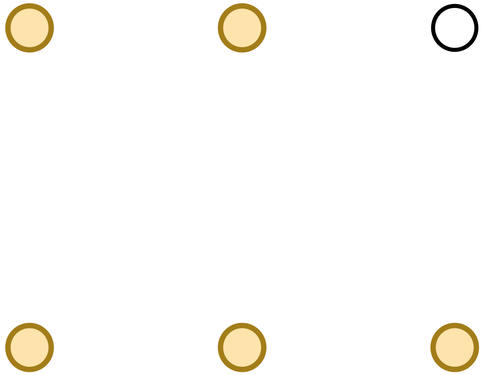
Generate a matching M in G_l and derive directed equality subgraph $G_{M,l}$



G



G_l



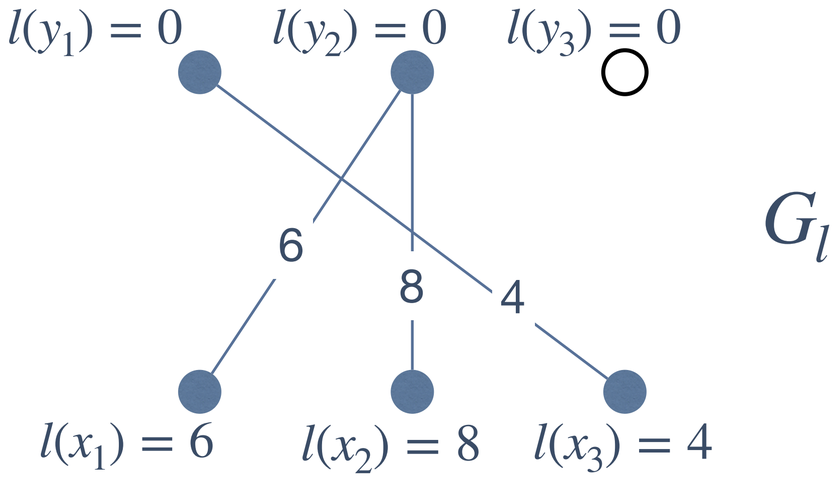
$G_{M,l}$

② Find an M-augmenting path in $G_{M,l}$

Use BFS similar to Hopcroft-Karp algorithm
- Start by enqueueing all free vertices in X.
- Stop when discovering some free vertex in Y.

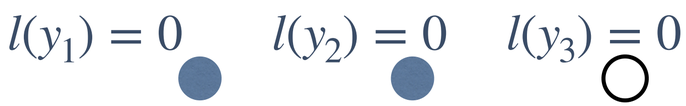
For a dense graph, this step takes

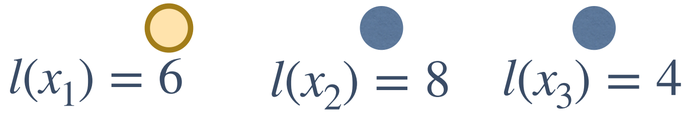
$$O(|V_l| + |E_l|) = O(|V| + |E|)$$
$$= O(2n + n^2)$$
$$= O(n^2)$$

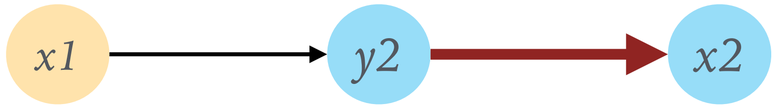


G_l

$G_{M,l}$

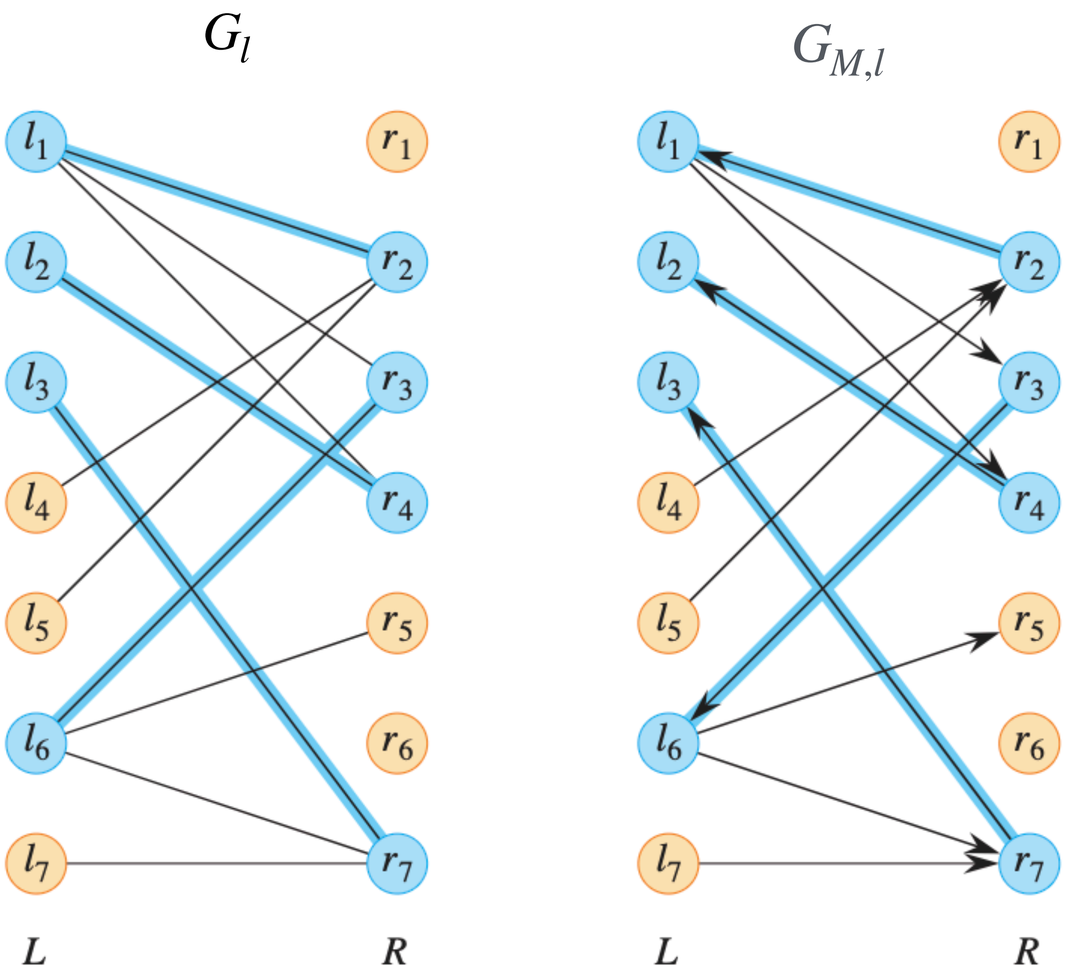


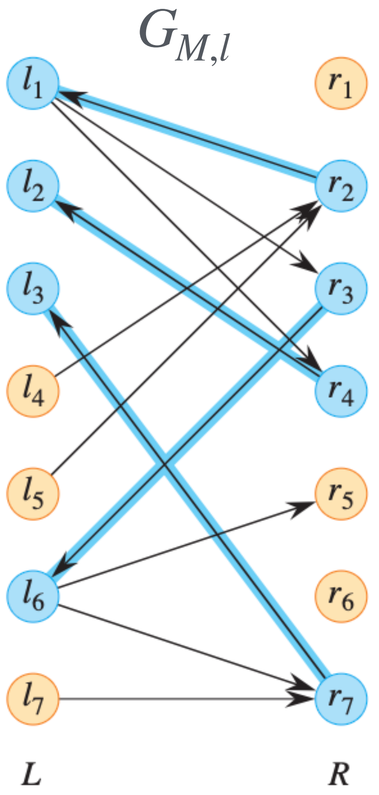




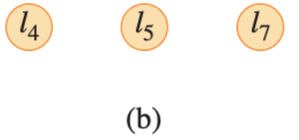
BFS forest $F = (V_F, E_F)$, $V_F = F_X \cup F_Y$

$$F_X = L \cap V_F = \{x_1, x_2\}$$
$$F_Y = R \cap V_F = \{y_2\}$$

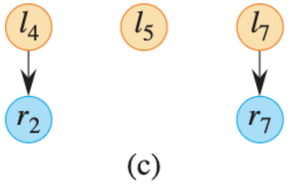

G_l
$G_{M,l}$
l_1
l_2
l_3
l_4
l_5
l_6
l_7
r_1
r_2
r_3
r_4
r_5
r_6
r_7
L
R



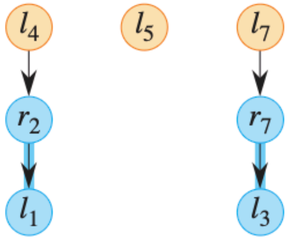
(a)



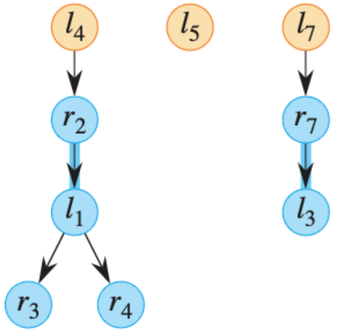
(b)



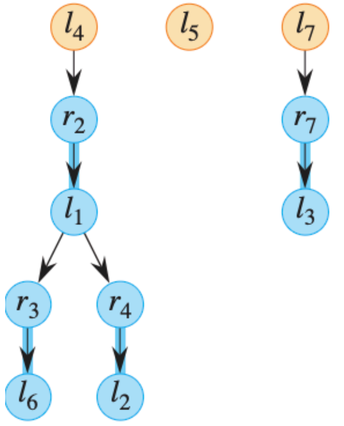
(c)



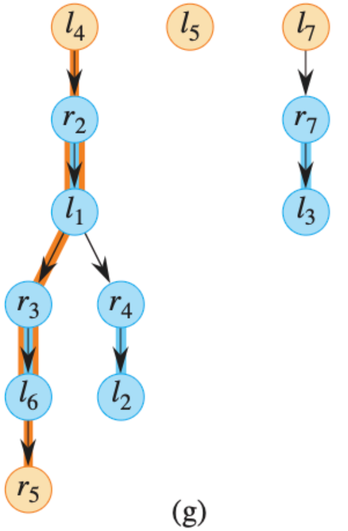
(d)



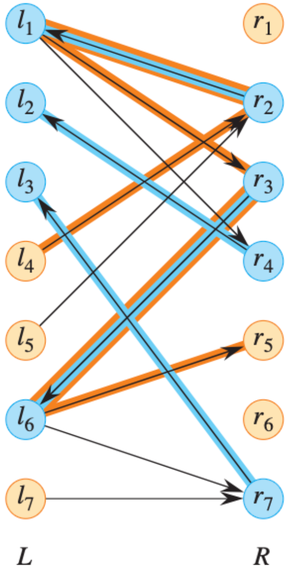
(e)



(f)



(g)

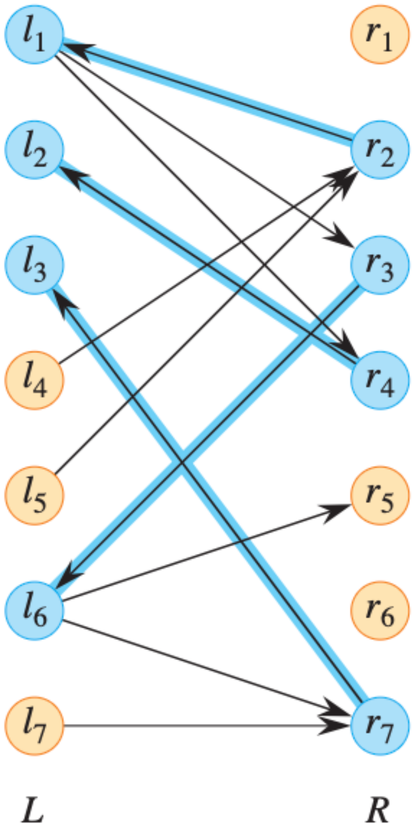


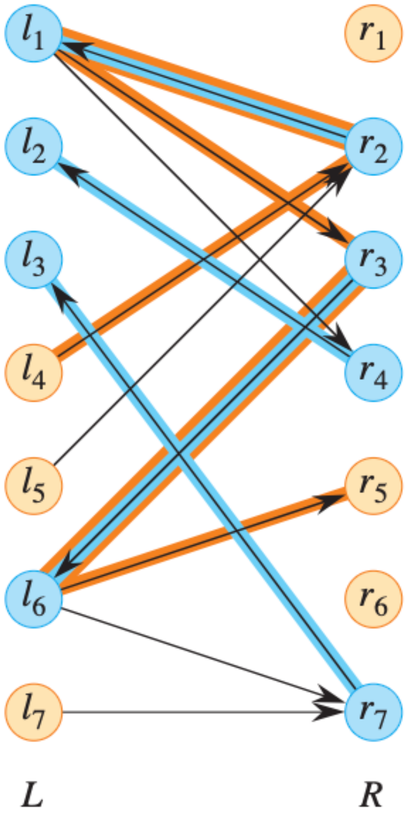
BFS forest $F = (V_F, E_F)$

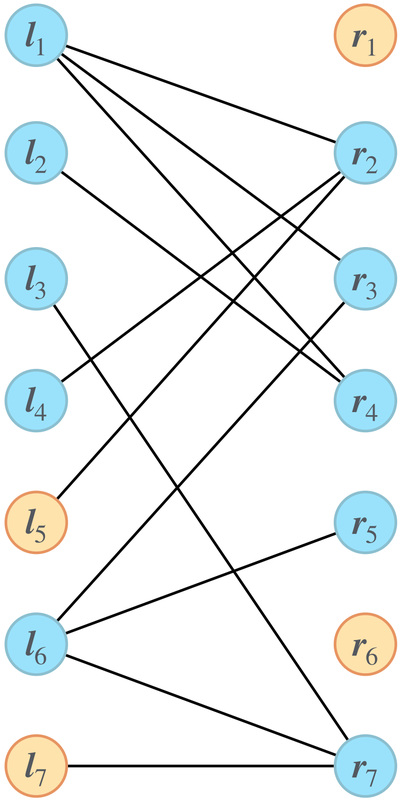
BFS procedure on $G_{M,l}$

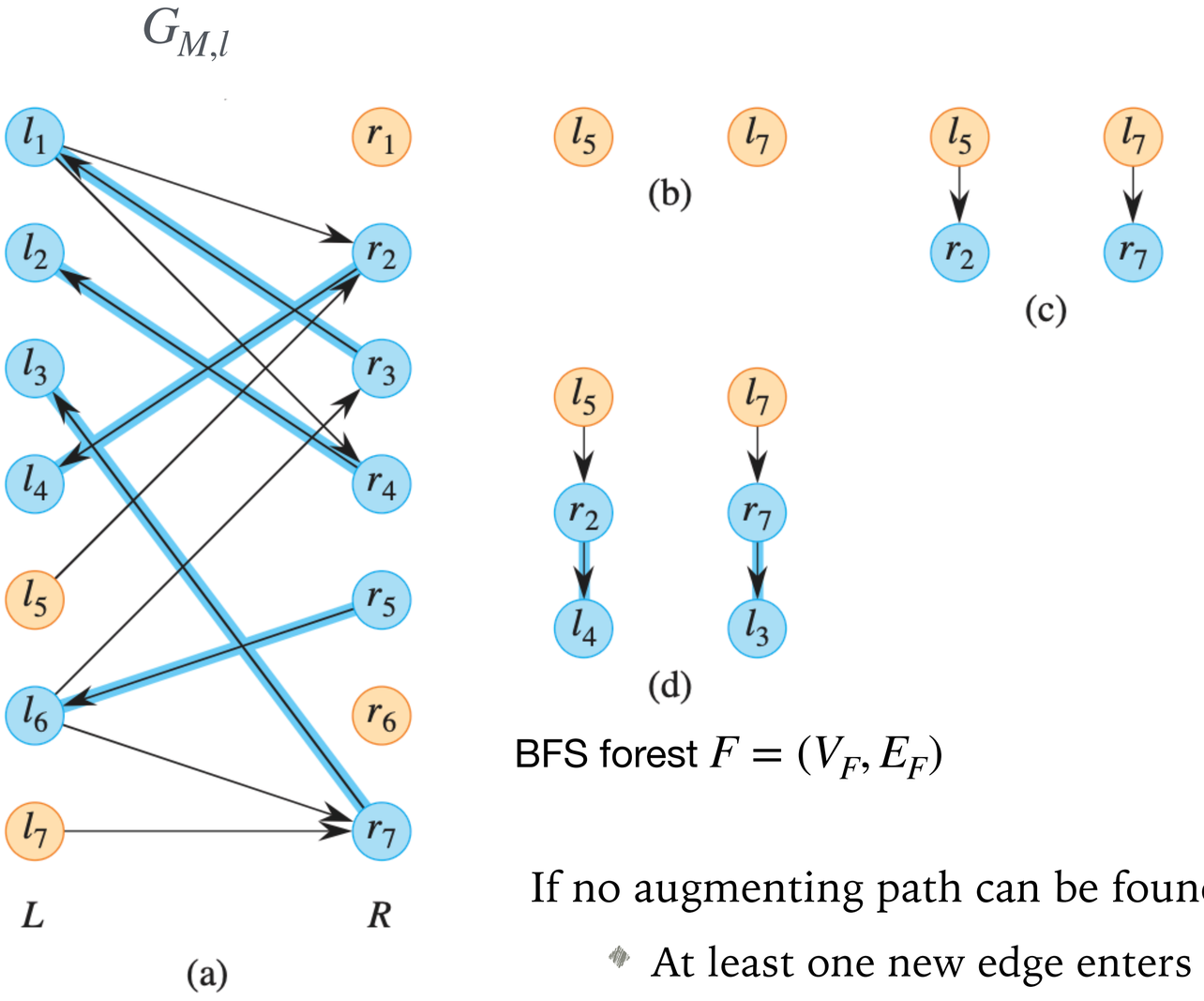
- Initialize the FIFO queue with all free vertices in L
- Each vertex only maintains one predecessor.
- Stop when reaching any free vertex in R.

Ref: Figure 25.5, pp.729.
T. Cormen et.al., Introduction to Algorithms, 4ed, MIT Press, 2022


l_1
l_2
l_3
l_4
l_5
l_6
l_7
r_1
r_2
r_3
r_4
r_5
r_6
r_7
L
R


l_1
l_2
l_3
l_4
l_5
l_6
l_7
r_1
r_2
r_3
r_4
r_5
r_6
r_7
L
R


l_1
l_2
l_3
l_4
l_5
l_6
l_7
r_1
r_2
r_3
r_4
r_5
r_6
r_7



BFS forest $F = (V_F, E_F)$

If no augmenting path can be found, modify the vertex labels so that

- At least one new edge enters G_l
- Any edge leaving G_l does not belong to M and $\boldsymbol{F}$

Ref: Figure 25.6, pp.730.
T. Cormen et.al., Introduction to Algorithms, 4ed, MIT Press, 2022

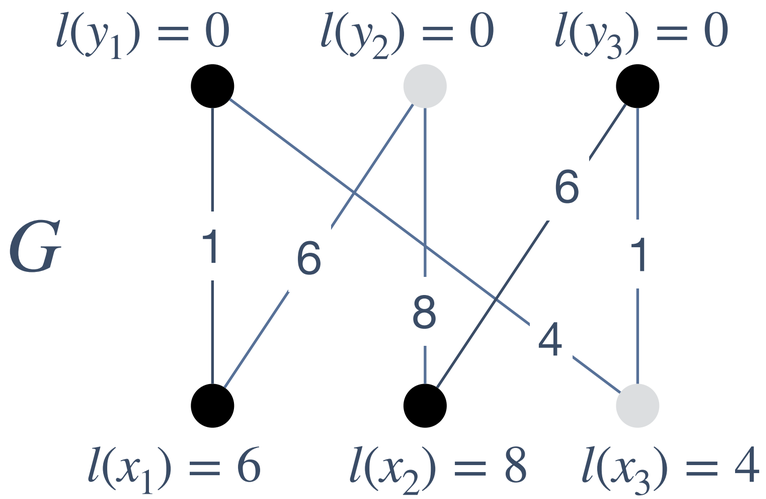
③ Improve labeling (1)

If no augmenting path can be found, modify the vertex labels as following:

3.1 Find the minimum of slack values

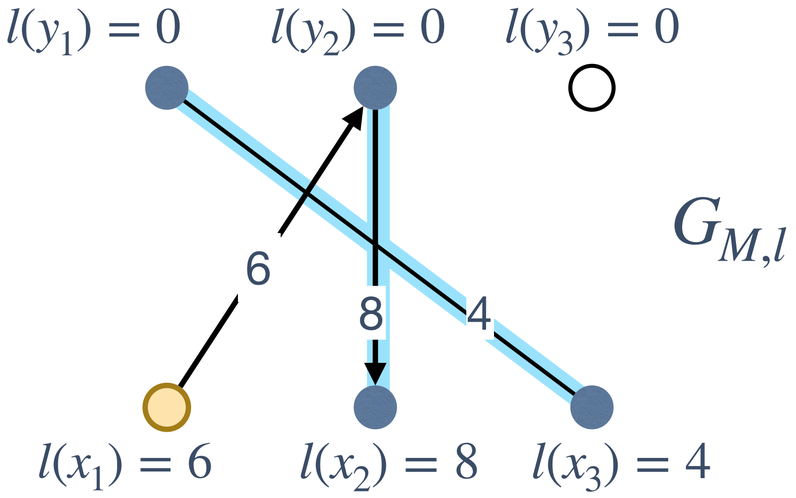
$$\delta = \min_{\substack{x \in F_X \\ y \in Y - F_Y}} l(x) + l(y) - w(x, y)$$

$$F_X = \{x_1, x_2\},\ Y - F_Y = \{y_1, y_3\}$$



$\delta =$

$$F_X = \{x_1, x_2\},\ F_Y = \{y_2\}$$

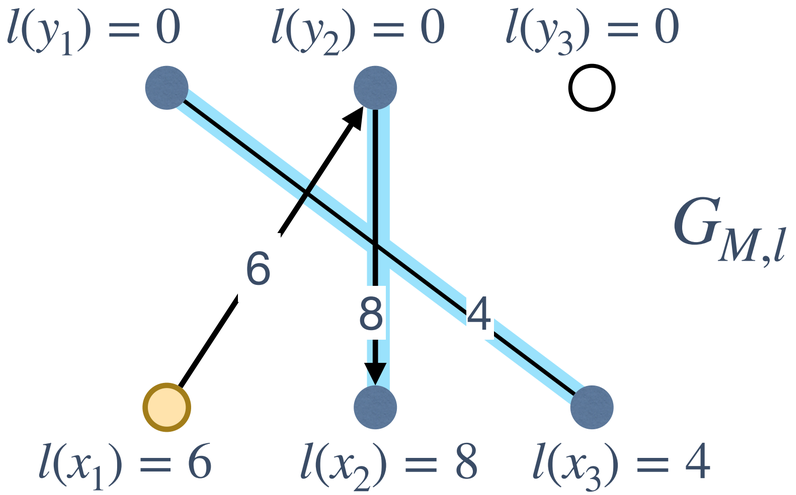


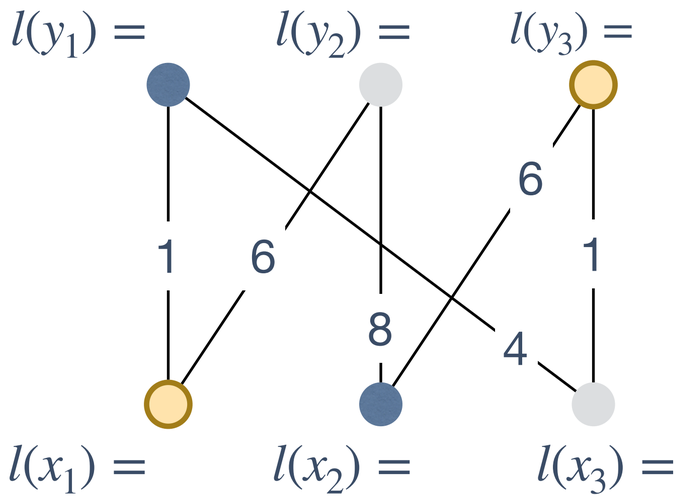
③ Improve labeling (2)

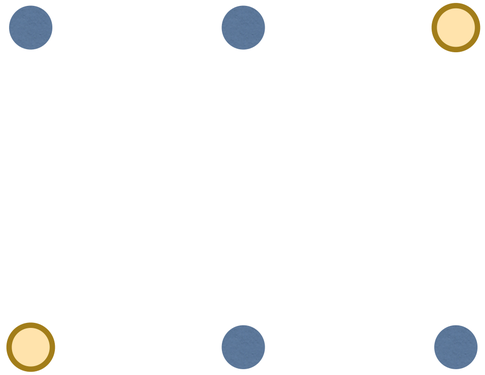
3.1 Find $\delta = \min_{\substack{x \in F_X \\ y \in Y - F_Y}} l(x) + l(y) - w(x, y)$

3.2 Set $$l(v) \leftarrow \begin{cases} l(v) - \delta & v \in F_X \\ l(v) + \delta & v \in F_Y \\ l(v) & \text{otherwise } (v \in V - V_F) \end{cases}$$

$F_X = \{x_1, x_2\}$, $F_Y = \{y_2\}$







Updated $G_{M,l}$

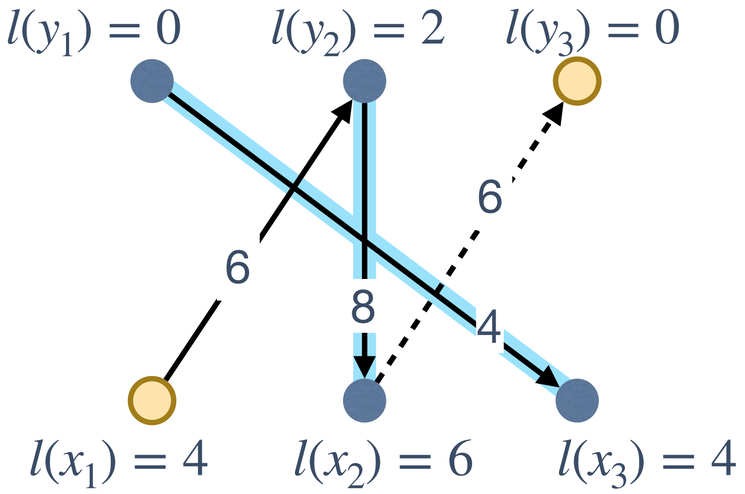
④ Add vertices for BFS from the updated $G_{M,l}$

For each new edge (x, y), $y \notin F_Y$

- If y is free, the augmenting path is found.
- Otherwise, enqueue y

Resume BFS until either

- an M-augmenting path is found or
- improve labels if the queue becomes empty before finding an M-augmenting path.



```
FIND-AUGMENTING-PATH(G_{M,h})
 1  Q = ∅
 2  F_L = ∅
 3  F_R = ∅
 4  for each unmatched vertex l ∈ L
 5      l.π = NIL
 6      ENQUEUE(Q, l)
 7      F_L = F_L ∪ {l}          // forest F starts with unmatched vertices in L
 8  repeat
 9      if Q is empty            // ran out of vertices to search from?
10          δ = min{l.h + r.h − w(l, r) : l ∈ F_L and r ∈ R − F_R}
11          for each vertex l ∈ F_L
12              l.h = l.h − δ    // relabel according to equation (25.5)
13          for each vertex r ∈ F_R
14              r.h = r.h + δ    // relabel according to equation (25.5)
15          from G, M, and h, form a new directed equality graph G_{M,h}
16          for each new edge (l, r) in G_{M,h}    // continue search with new edges
17              if r ∉ F_R
18                  r.π = l                        // discover r, add it to F
19                  if r is unmatched
20                      an M-augmenting path has been found
                            (exit the repeat loop)
21                  else ENQUEUE(Q, r)             // can search from r later
22                      F_R = F_R ∪ {r}
23      u = DEQUEUE(Q)                             // search from u
24      for each neighbor v of u in G_{M,h}
25          if v ∈ L
26              v.π = u
27              F_L = F_L ∪ {v}                    // discover v, add it to F
28              ENQUEUE(Q, v)                      // can search from v later
29          elseif v ∉ F_R                         // v ∈ R, do same as lines 18–22
30              v.π = u
31              if v is unmatched
32                  an M-augmenting path has been found
                        (exit the repeat loop)
33              else ENQUEUE(Q, v)
34                  F_R = F_R ∪ {v}
35  until an M-augmenting path has been found
36  using the predecessor attributes π, construct an M-augmenting path P
        by tracing back from the unmatched vertex in R
37  return P
```

Ref: pp.738.
T. Cormen et.al., Introduction to Algorithms, 4ed, MIT Press, 2022

In-Class Assignment L7-1 (Group of 3 or less): Maximum-Weight Matching

(25 minutes) Submit to LEB2

You are holding a party and you want a musician to perform, a chef to prepare food, and a cleaning service to help clean up after the party. Three companies provide each of these three services, but one company can only provide one service at a time (i.e., Company B cannot provide both the cleaners and the chef). You are deciding from which company you should purchase each service to minimize the cost of the party. Each company charges the amount shown in the table below (in hundreds of dollars).

(a) Construct a weighted complete bipartite graph that models this problem as the maximum-weight matching.

(b) Execute lines 4-5 in the Kuhn-Munkres algorithm by showing feasible labeling of the graph and the corresponding equality subgraph.

(c) Execute the while loop (lines 7-11) in the Kuhn-Munkres algorithm ***for one iteration*** by showing how the labels are improved and the equality subgraph is updated.

Company	Cost for Musician	Cost for Chef	Cost for Cleaner
Company A	11	12	15
Company B	15	13	18
Company C	12	15	25

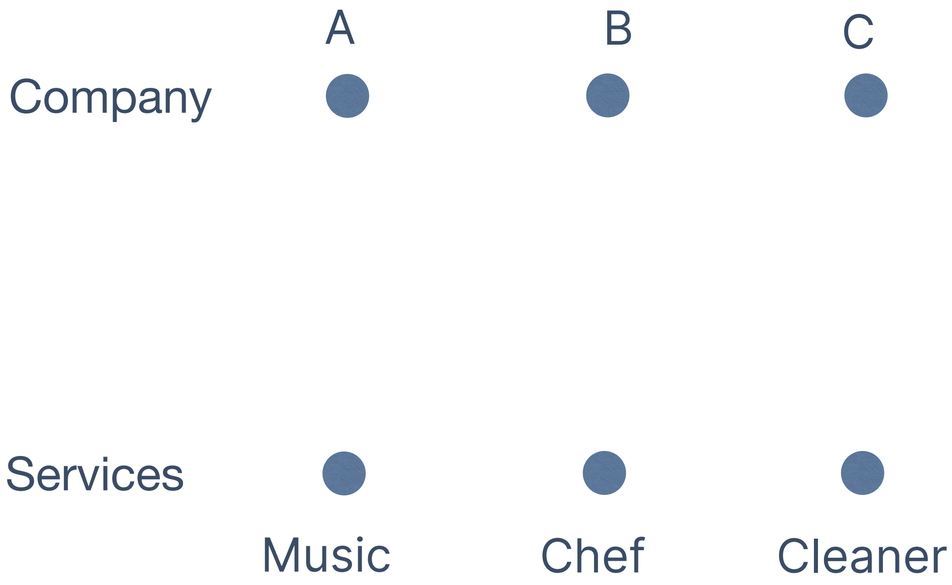
Algorithm Hungarian (Kuhn-Munkres)

Input: A weighted complete bipartite graph $G = (V, E), V = X \cup Y, |X| = |Y| = n$
Output: A maximum-weight perfect matching M in G

Obtain G_l from G by using default vertex labeling l.
Obtain any arbitrary matching M (e.g., empty matching, greedy matching)

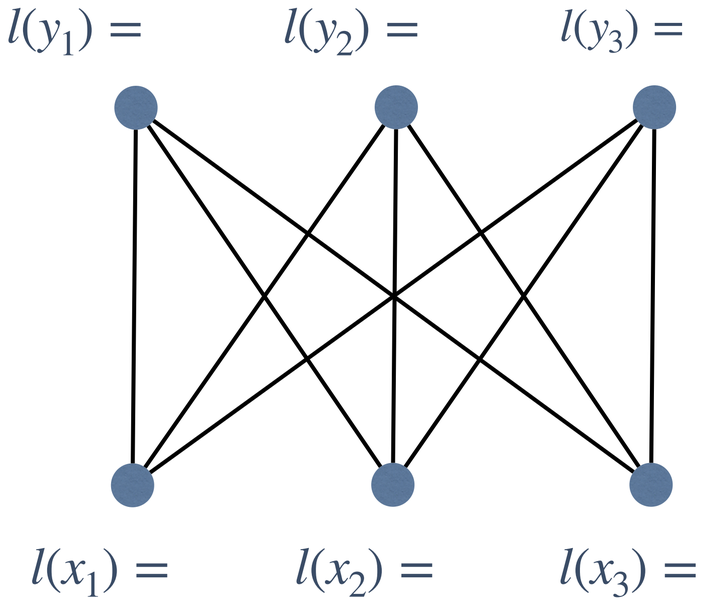
while M is not perfect matching in G_l **do**
    Find an M-augmenting path in G_l to improve M.
    if M-Augmenting path in G_l does not exist **then**
        Improve the labeling
        Update G_l
return M

(a) Weighted complete bipartite graph of the problem

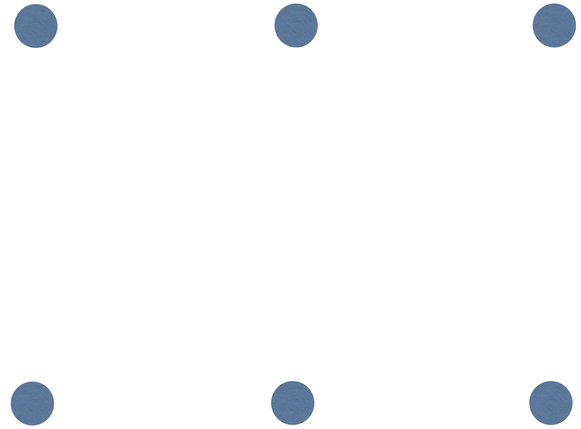


(b) Feasible Labeling and Equality Subgraph

G

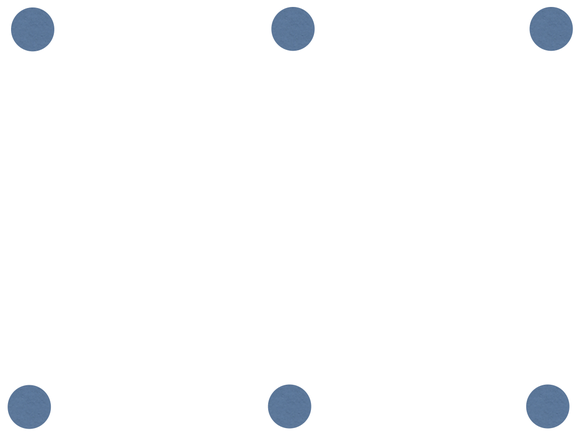


G_l and Initial matching M

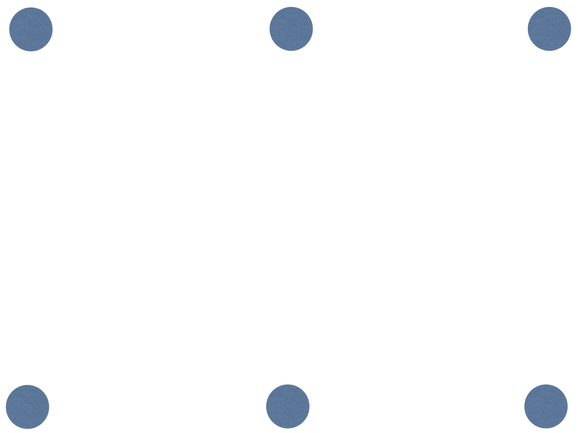


(c) BFS Forest on the Equality Subgraph

Equality subgraph G_l



Directed equality subgraph $G_{M,l}$



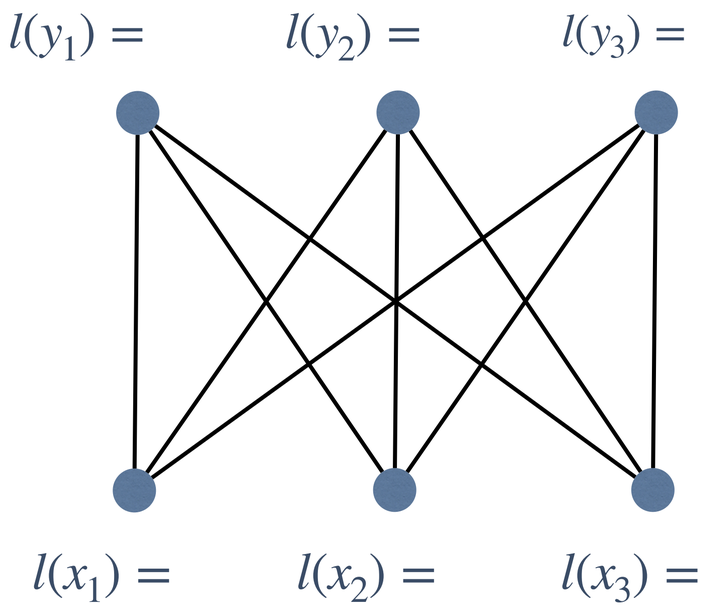
BFS forest

$F = (V_F, E_F),\ V_F = F_X \cup F_Y$

$F_X =$

$F_Y =$

(c) Updated Labels



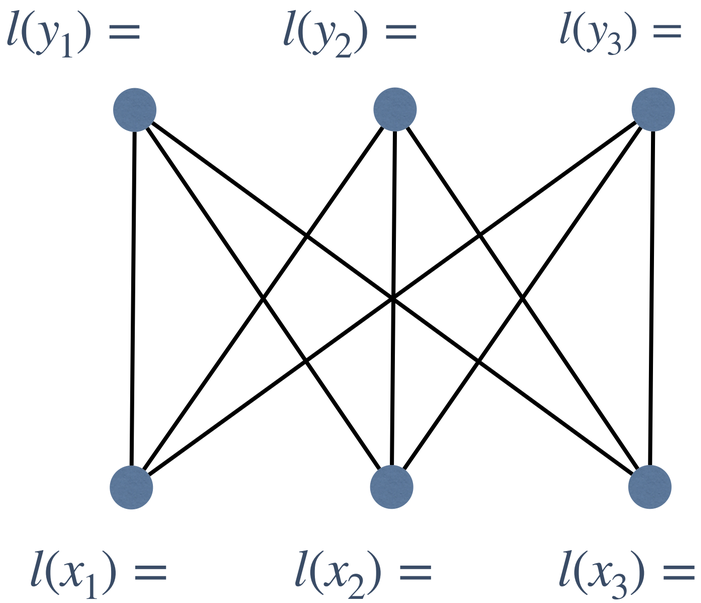
$F_X =$

$F_Y =$

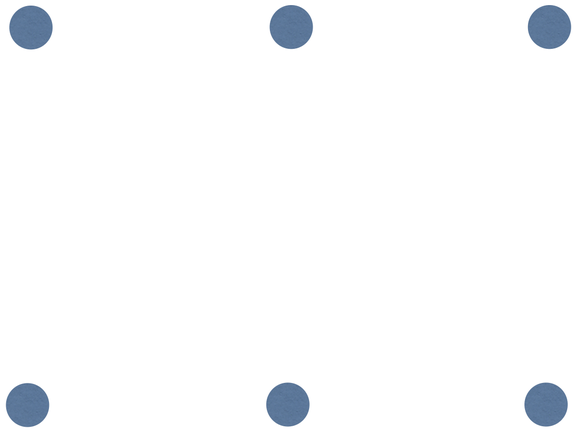
$$\delta = \min_{\substack{x \in F_X \\ y \in Y - F_Y}} l(x) + l(y) - w(x, y)$$

$=$

(c) Updated Equality Subgraph

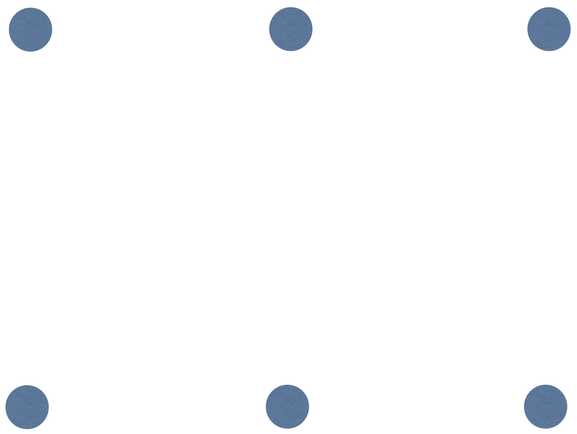


Updated equality subgraph G_l



(c) BFS Forest on the Updated Equality Subgraph

Updated directed equality subgraph $G_{M,l}$



BFS forest

$F = (V_F, E_F),\ V_F = F_X \cup F_Y$

$F_X =$

$F_Y =$

Scenario: Survey Design

Each customer is asked about a subset of products he/she has purchased, not too few and not too many.

Sufficient number of customers must be asked for each product, not too few and not too many.





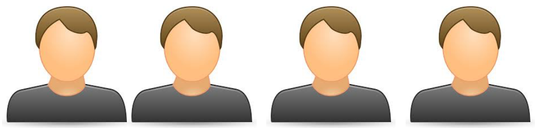
Being asked questions about a subset of products (not too few or too many).

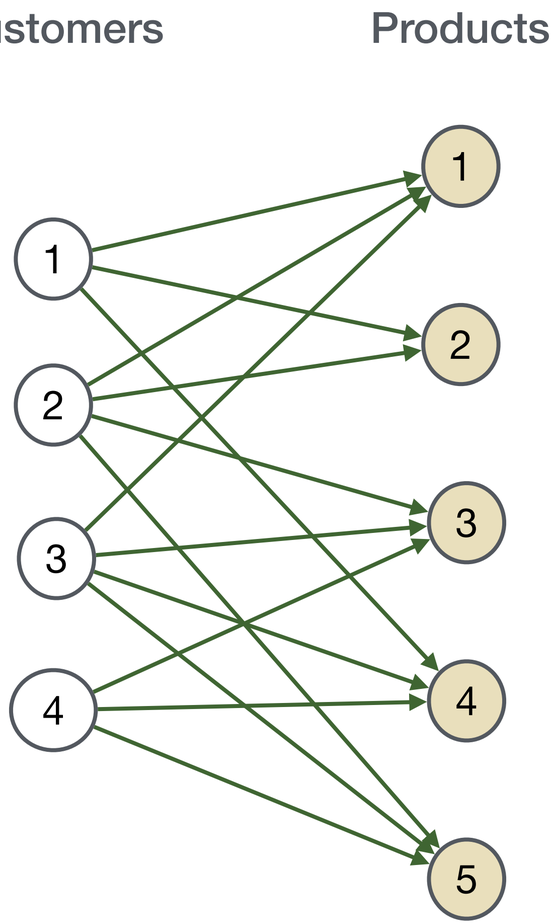




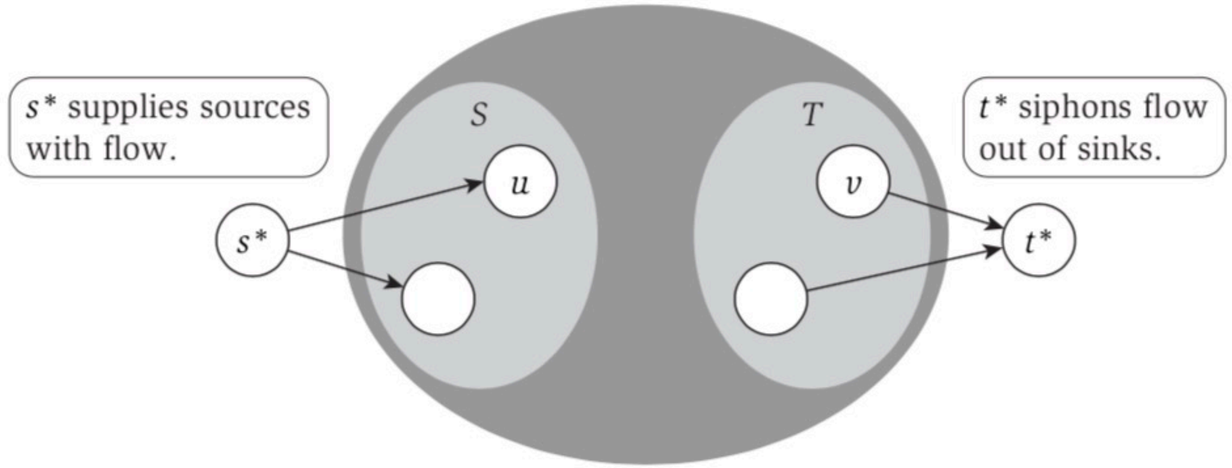
Distinct # customers are asked about each product. (not too few or too many).

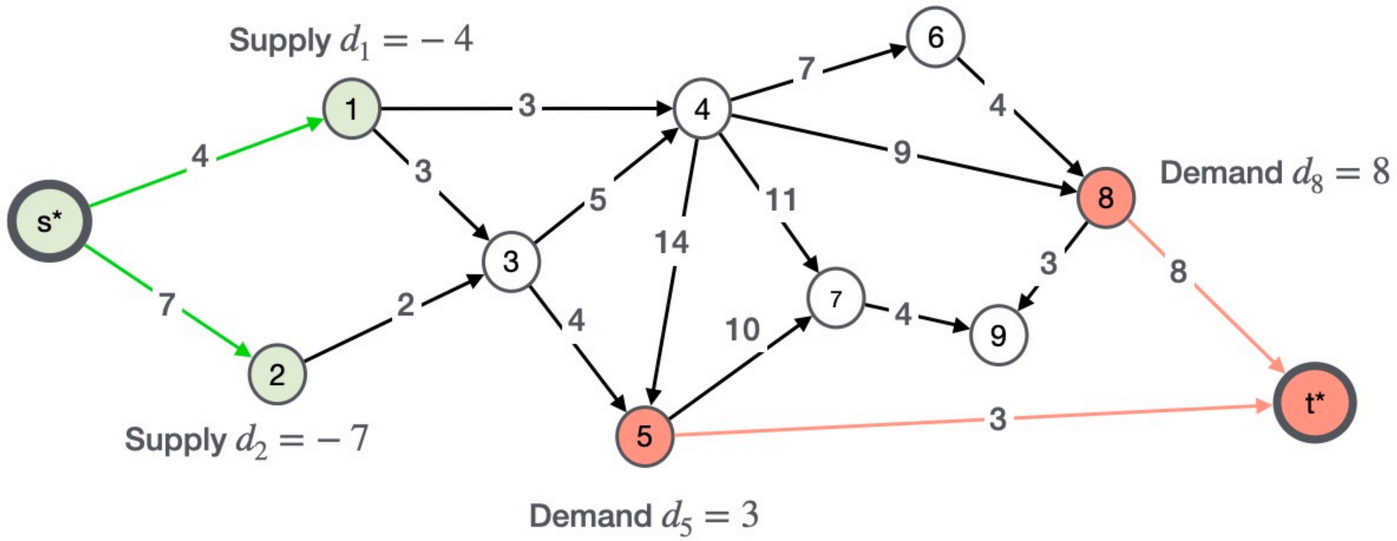





Customers
Products
1
2
3
4
1
2
3
4
5

Review: Circulation Problem

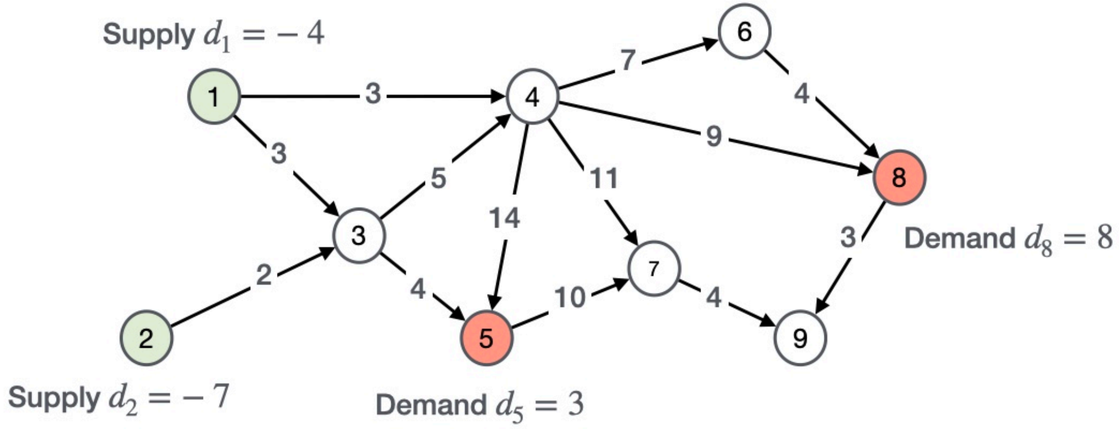




Circulation with Lower Bounds

Given a circulation network G with

- A set of supply nodes S with supply $d_v < 0$, $\forall v \in S$
- A set of demand nodes T with demand $d_v > 0$, $\forall v \in T$
- Lower bound l_e, $0 \leq l_e \leq c_e$, $\forall e \in E$
- Remaining nodes with $d_v = 0$ (Transhipment node)



Determine a circulation f with

- **Capacity conditions**: Assignment of $f(e)$ such that $l_e \leq f(e) \leq c(e)$ for $\forall e \in E$
- **Demand conditions:** Requirement at node v:

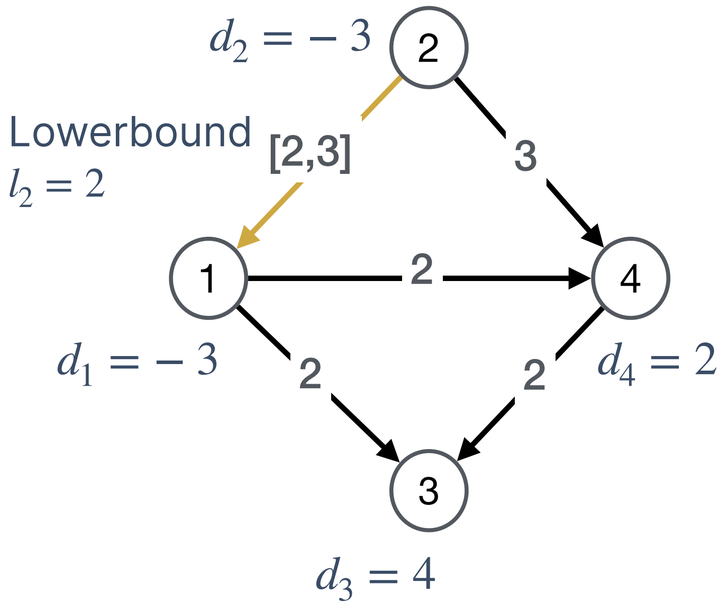
$$\sum_{e \text{ into } v} f(e) \; - \sum_{e \text{ out of } v} f(e) = d_v$$

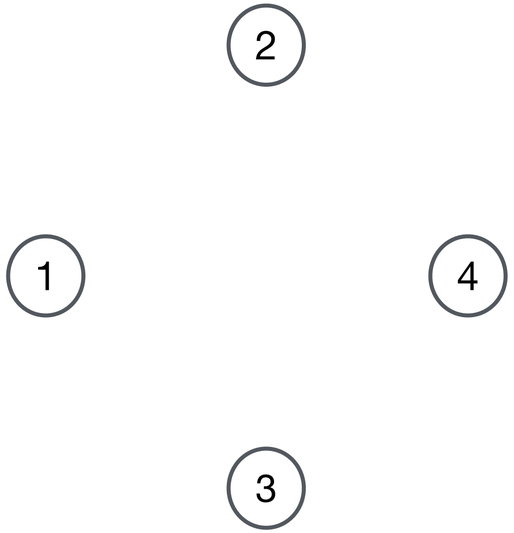
Lowerbound Elimination

Determine the ***net lower-bound*** at v: $L_v = \sum_{e \text{ into } v} l_e - \sum_{e \text{ out of } v} l_e$

Create circulation network $\boldsymbol{G}'$ with edge capacity $c'(e) = c(e) - l_e$ with zero lowerbound, and $d'_v = d_v - L_v$

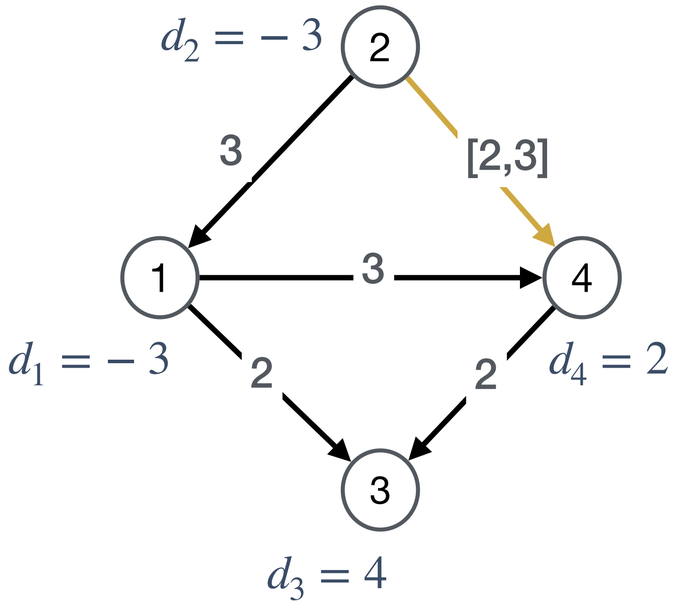
Feasible circulation $f'(e)$ in $\boldsymbol{G}'$ implies feasible circulation $f(e) = f'(e) + l_e$ in $\boldsymbol{G}$, or vice versa.



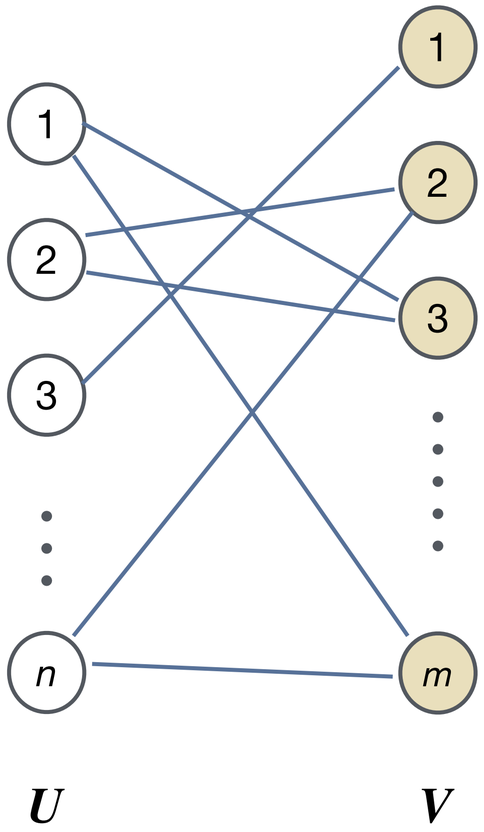


Self Test: Circulation with Lower bounds

https://forms.office.com/r/1y0Pu7xfKP?origin=lprLink



One-to-Many Maximum Bipartite Matching with Constraints



Each vertex $i \in \boldsymbol{U}$ connects and can be assigned to many vertices $j \in \boldsymbol{V}$

Goal: Find the max-cardinality matching

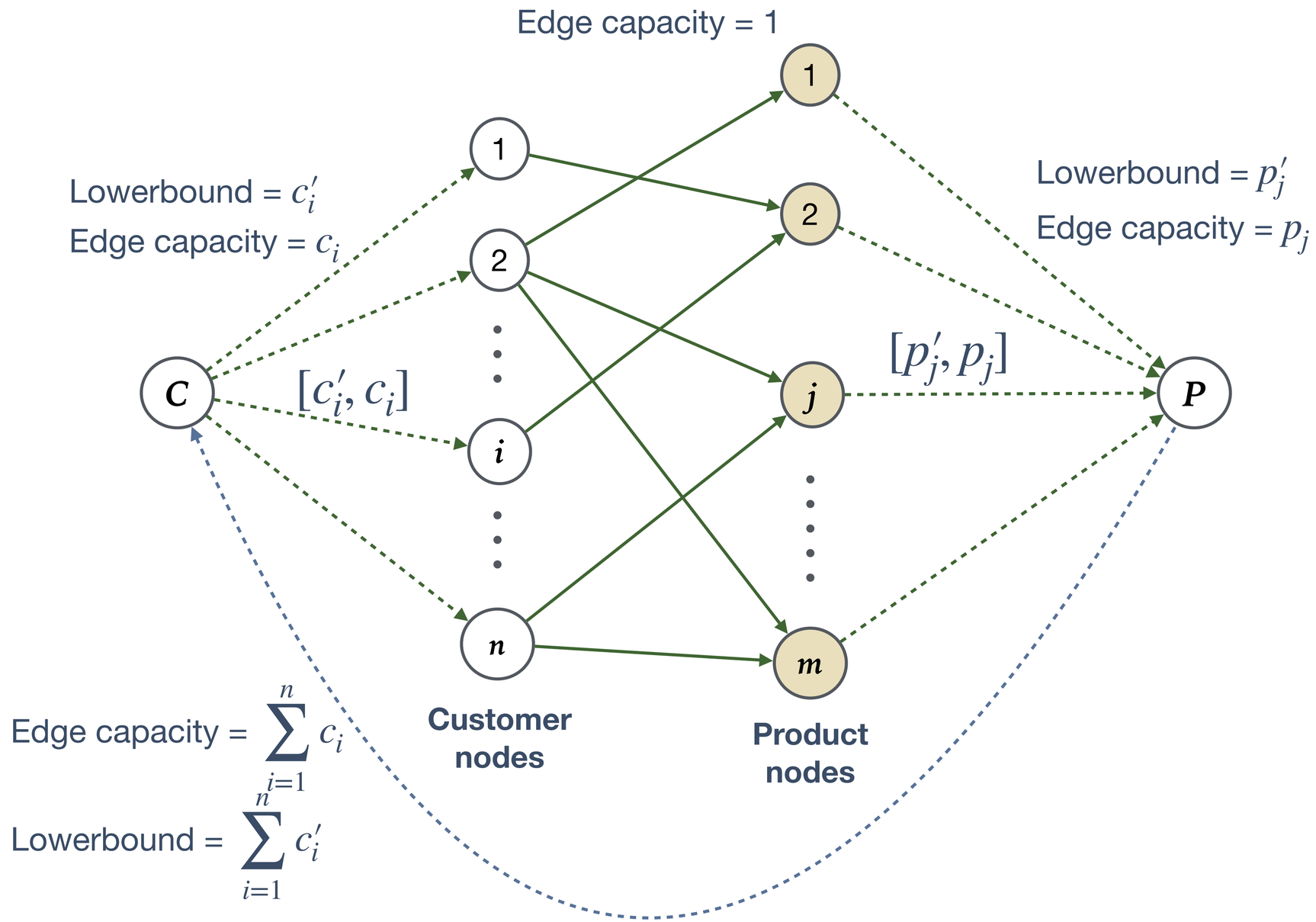
Constraints:

Each vertex $i \in \boldsymbol{U}$ must be assigned to # vertices between c_i' and c_i.

Each vertex $j \in \boldsymbol{V}$ must be matched to # vertices between p_j' and p_j.

Circulation Network with Lowerbound

All the nodes have zero demand.



In-Class Assignment L7-2 (Group of 3 or less): Survey Design

(25 minutes) Submit to LEB2